A Multi-Agent Approach to Validating Systems Models

Qiana Curcuru
MIT Lincoln Laboratory
244 Wood St.
Lexington, MA, 02421
Qiana.Curcuru@ll.mit.edu

Ethan Cox
MIT Lincoln Laboratory
244 Wood St.
Lexington, MA, 02421
ethancox@iastate.edu

DISTRIBUTION STATEMENT A. Approved for public release. Distribution is unlimited. This material is based upon work supported by the Department of the Air Force under Air Force Contract No. FA8702-15-D-0001 or FA8702-25-D-B002. Any opinions, findings, conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the Department of the Air Force.
© 2026 Massachusetts Institute of Technology. Delivered to the U.S. Government with Unlimited Rights, as defined in DFARS Part 252.227-7013 or 7014 (Feb 2014). Notwithstanding any copyright notice, U.S. Government rights in this work are defined by DFARS 252.227-7013 or DFARS 252.227-7014 as detailed above. Use of this work other than as specifically authorized by the U.S. Government may violate any copyrights that exist in this work.

Copyright © 2025 by the author(s). Permission granted to INCOSE to publish and use.

Abstract

The release of SysML v2 introduces new opportunities to leverage AI for system modeling. Agentic AI, a framework where language models autonomously execute tasks with external tools and information, offers a novel approach for SysML v2 modeling and validation.

This paper explores the architecture and methodology for developing a multi-agentic Model Based Systems Engineering (MBSE) tool to assist with model validation, focusing on accuracy, efficiency, and scalability while also addressing concerns such as cost, explainability, data control, and security.

This body of work proposes a novel framework for SysML v2 model validation, specifically focusing on requirement derivation and model relationships. This work also addresses common deployment challenges. The modularity of this framework will be expanded to address additional validation objectives and design recommendations to create a full end-to-end agentic modeling tool. This framework demonstrates promising validation results on a simplified SysML v2 CubeSat model. Further testing on larger, real-world models, and integration with industry standard MBSE tools such as MagicDraw is suggested to fully assess its scalability and broader applicability.

To implement this approach, objective modeling rules were distilled into smaller functions and assigned to agents within an agentic AI tool. The agentic tool was then tested on a simplified SysML v2 CubeSat model to evaluate its validation capabilities. A similar test was then given to four systems engineers (two junior engineers, and two senior engineers). In addition to being approximately 10 times faster than the fastest engineer at finding parent-child relationships, the results demonstrated that the agentic tool had a better average correctness score compared to systems engineers. When only considering the senior engineers, the agentic tool had a comparable average correctness score.

Keywords

Artificial Intelligence, Agentic AI, SysMLv2, Model Validation, Secure Deployment, LLM—Large Language Model, MBSE—Model-based Systems Engineering, SLM—Small Language Model

Introduction

MBSE models have transformed systems engineering by providing a structured approach to managing systems. SysML v2 resolves many of the limitations of SysML v1 by enhancing support for automation and AI in systems modeling through a code-based model architecture (Object Management Group, 2025). Despite its advantages, SysML v2 model validation is still challenging due to the complexity and scale of the models that systems engineers work with every day. This paper investigates the use of agentic AI for SysML v2 model validation, focusing on its accuracy, efficiency, and scalability. While model validation is a broad concept, this initial study focuses on requirement derivation and relationships. Requirement analysis is a manual task that requires meticulous review and attention to semantic details, which is why these two challenges were chosen to be tackled first. While related works have investigated the use of AI for system model generation, there is a significant gap in the literature for work that addresses using agentic AI for SysML v2 model

validation. This paper also evaluates an agentic AI framework for SysML v2 model validation, focusing on requirement derivation and model relationships, by comparing its accuracy and efficiency against engineers using a simplified CubeSat model, demonstrating encouraging initial results, and identifying areas for improvement.

Literature Review

This work builds on existing research in model validation and AI for SysML. While tools like MagicDraw address syntactic issues, they would benefit from a tool that could interpret semantic nuances that understand the meaning of language. Additionally, related AI work in system model generation and validation has the potential to be deployed in secure production environments if AI models could be run on-premises (the ability for hardware and software to be located on an organization's own physical property (CDW, 2023)).

Model validation is challenging, as existing frameworks often rely on subject matter experts to define correctness and lack automation for semantic evaluation (Moody, 2005). For example, the proposed framework MagicGrid offers details on required model elements and relationships for different aspects of a program, but still relies on manual, human expertise for implementation (Morkevičius, 2022).

MagicDraw has built in validation tools that validate certain aspects of models. However, they primarily address syntactic issues and are limited in their ability to address semantic nuances, requiring human interpretation (Sultan et al. 2024).

Related work exists on generating models with commercial LLMs such as work done by Johns, B. et al. (2024), where AI demonstrated that reasonable MagicDraw diagrams are produced from a language prompt. The work by Sultan et al. (2024) initiated the use of commercial LLMs for model validation. This work integrated an existing model validation tool to assist in generating feedback queries for the LLM to correct. However, neither body of work addresses secure deployment, creating a substantial challenge when data privacy is important. These works also did not address the user's lack of control or explainability of the results that were produced. The only influence the user had for model creation and modification was enhanced prompt engineering with no understanding of why the model made certain design decisions.

Additionally, the size of MBSE models for AI is challenging. The work by Sultan proposes a potential solution to the limited context window (the amount of text, in tokens, that the model can consider or *remember* at any one time (IBM, n.d.)) of conventional LLM chatbots to breaking down complex models into sub-models as planned future work, but excludes information as to how it would be implemented in practice (Sultan, 2024).

Work by Ludovico et al (2020) uses reinforcement learning techniques to correct and fix system models. While results were promising, this work only addresses syntactic issues such as duplicate attributes, self-inheritance, bounds inconsistencies, and other issues of similar nature. The body of work described in this paper extends using AI methods for syntactic correction by incorporating semantic correction and semantic evaluation capabilities.

Agentic AI is showing encouraging results for its ability to aid in the automation of complex tasks for a fraction of the cost of human workers. The study performed by Wang et al. (2025) evaluated the performance of AI agents compared to human workers in five categories and found that agents were 88.3% faster and cost 90.4%-96.2% less than their human equivalents (Wang et al., 2025).

Proposed Solution

This work demonstrates AI for model validation by breaking down objective modeling rules into smaller functions integrated into an agentic AI tool. Backed by language models, AI agents' reason, plan, and adapt. An agentic framework can involve one to multiple agents, each assigned a specific objective to accomplish. A multi-agentic framework can accomplish complex tasks by having agents work together. This also allows the systems engineers to steer the AI model in the right direction by providing domain specific knowledge to the model. Agents also have access to tools that can interact with data outside of itself such as executing code or other external programs. This combination of probabilistic and deterministic methods allows for a greater capability than a language model alone and also offers more transparency and control over the agents' decisions (Huizenga & Yang, 2025). This tool addresses common secure deployment concerns, while also providing a level of explainability to its responses so there is a level of transparency as to why the tool made a decision. The user also defines specific decision criteria for the agents to follow giving greater user-control over the agents' decisions based on their semantic evaluation. This semantic capability can be combined with conventional function calls to provide syntactic validation too.

Each agent is composed of its own on-premises (on-prem), small language model (SLM) that serves as the brain of the agent. SLMs were chosen for their ease of deployment and ability to be hosted locally, supporting data security and privacy. Their small size also reduces infrastructure and development costs without sacrificing performance. (Belcak et al., 2025). In this body of work, Mistral 7B was selected for its size and proven performance on commonly used benchmarks used for evaluating language models (Mistral, 2023).

This framework utilizes four agents, an orchestration agent, a metrics agent, a requirement agent, and a relationship agent. When the user makes an initial query, an orchestration agent determines which agent(s) are needed to complete the task.

The model metrics agent is assigned the task of querying the model to answer questions, and respond to the user directly, or engages either the relationship or requirement agent to get the information it needs to provide the user a response. For example, the model metrics agent finds orphaned requirement(s) in the model and either provide that output to the user directly, or pass its output to the requirement agent or the relationship agent depending on the original query.

The primary benefit to the model metrics agent is that it has access to the SysML v2 model via the agentic tool's graph database. Inserting the system model into a graph database addresses the context window limitations of conventional LLM chatbots, which cannot effectively process large system models. Graph databases store data as relationships (Neo4j, 2025), mirroring the structure of SysML v2 models. They provide a comprehensive representation of model elements and their relationships to requirements, part objects, and behaviors. Refer to Figure 1 for a visual representation of a graph database. In this example, the requirements, part objects, and behaviors of a satellite consisting of a data processing unit (DPU), random access memory (RAM), and a solid-state drive (SSD) are shown.

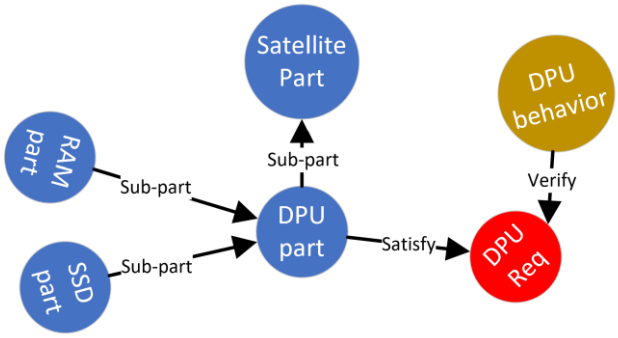


Figure 1 Visualization of Graph Database

The requirement agent is assigned the task of finding parents for orphaned requirements, and the relationship agent finds model objects that may satisfy requirements. Both agents use semantic analysis to identify missing relationships and provide recommendations to the user, such as parent-child relationships for orphaned requirements (requirement agent), or connections between requirements and model elements like behaviors and part objects (relationship agent). Both agents also consider the existing model state when making decisions.

The approach proposed in this paper decreases modeling time while increasing model integrity by identifying and implementing missing model relationships as well as addressing gaps in SysML v2 models. This framework is modular, allowing additional agents to be added to address other modeling challenges such as finding incompatible interfaces, evaluating the quality of requirements, answering more complex model queries, and more. Existing agents evolve to become more advanced over time, and new agents can be introduced, as additional validation and design rules are defined, supporting an end-to-end agentic tool.

Methodology

Agent Design & Architecture: Refer to Figure 2 for a depiction of the agentic architecture described in this section.

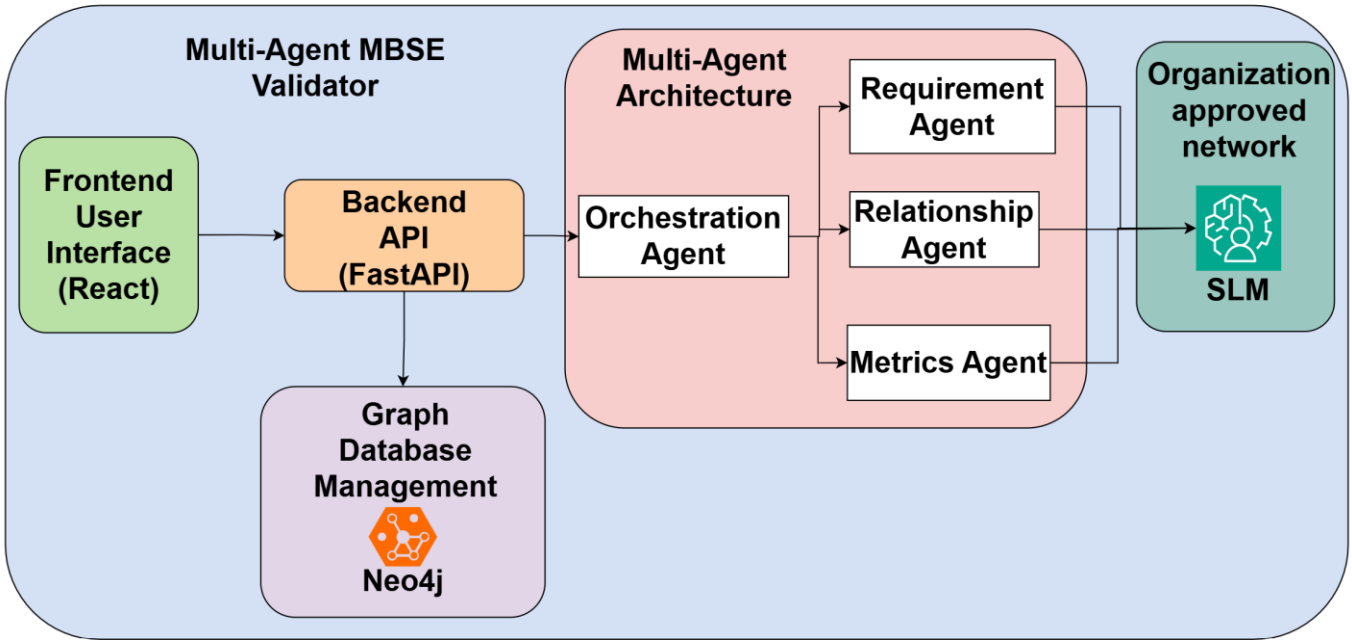


Figure 2. Depiction of Agent Architecture

Agent Framework: This utility uses LlamaIndex, a modular open-source framework for developing and deploying AI agents (LlamaIndex). This architecture allows the user to provide a query to a single orchestration agent that deploys and delegates tasks which are written to accomplish specific tasks (Huizenga & Yang, 2025). If speaking to an AI chatbot directly, the user has little control over how the AI makes a decision beyond rewording the prompt. Conventional AI chatbots struggle to search through large models and make reasonable decisions due to limitations in context window and reasoning capacity. Without specialized retrieval strategies, conventional AI chatbots cannot capture all dependencies, hierarchies, and interactions effectively (Huizenga & Yang, 2025).

Agentic architectures enable developers to instruct agents to search models for relevant information and apply defined decision criteria, breaking complex reasoning into smaller, more manageable tasks.

On-Prem Deployment: It is estimated that using SLMs on the order of 7B parameters is 10-30x cheaper in latency and energy consumption compared to larger commercial LLMs like ChatGPT-4o (Belcak et al., 2025). Their size allows them to be hosted locally, on prem, enhancing data security and privacy, and enabling their use in both proprietary, unclassified applications, as well as classified environments.

Capability: Each agent can utilize tools and other agents to accomplish tasks. Tools may come in the form of software API calls, database queries, internet search calls, or others. The agent may use its discretion on which other tools or agents to employ in order to complete its assigned task. In this case, a *tool* is a capability that allows the AI agent to execute tasks that may involve communicating and interacting with elements and information outside of itself so that it has capabilities beyond just providing basic language generation. (Huizenga & Yang, 2025). The tools made available to the agents in this paper include Python function calls, cypher queries, and other agents. By combining deterministic and probabilistic methods, the user gains reliability and control over the outputs.

Advanced Search and Retrieval: There are three main components of the search and retrieval component of this agentic tool: a Python library for parsing into the database, a database application, and a prewritten query library. A set of Python functions were developed to parse SysML v2 code into a graph database. This agentic tool utilizes the Neo4js graph database to store all model relationships and objects. The graph database can also be hosted on-prem to maintain data security (Neo4j, 2025). Parsing the model into a graph database enables efficient querying of the system model's constituent parts. The model metrics agent can then utilize a library of pre-written queries to answer user queries efficiently. This also provides the SLM with only the relevant parts of the model to make decisions, rather than the entire model at once. This approach overcomes the conventional context window limitation of conventional LLM chatbots.

For example, to determine if a requirement is satisfied by a part object, the graph database is queried to get a list of requirements and a list of model parts. Then, the AI model is only provided one requirement and one

model element at a time to make a semantic comparison. This lessens the compute required when compared to ingesting an entire model at once. By ensuring that only relevant model components are queried, the framework promotes consistent and reliable decision-making.

Consistency and Reliability: The benefit of using tools in this body of work is that they provide consistent, repeatable, and reliable behaviors in task execution by giving the user control over the decision-making criteria, enhancing explainability. In the same way that systems engineers create models by reasoning through sets of guidelines from memory, these best practices can be broken down into smaller constituent tasks, each distilled into an agent through a series of sequential or parallel function calls. These calls are executed when specific situational circumstances are met. By providing agents with a library of predefined function calls and specific criteria for making decisions, the framework promotes consistent and reliable task execution, improving the overall validation process. For example, none of the evaluated language models understood SysML v2 at the time of development. Python function calls were used to enable users to automatically implement the agent's recommendations within SysML v2 files via the agentic tool. This ensures that code execution will be performed consistently and reliably.

Agent Use Cases

Decomposition: This agentic tool was designed by breaking down larger tasks into their smaller constituent tasks, just as a SME would think through them. For instance, a complex concept such as establishing relationships for requirements that are not satisfied by any model objects is divisible into distinct agents, such as identifying which skillsets are needed, identifying a requirement, and evaluating which elements form a relationship with that requirement. Therefore, three agents were designed to accomplish this objective, each assigned to one of the three tasks described. The subjectivity of understanding and comparing requirements is one of the more challenging tasks. This is a test of language comprehension, requiring a deep understanding of both the system, and the world in general. This task's reliance on semantic evaluation makes it perfectly suited for a language model to address.

Integration: By decomposing a large goal into individual tasks, the agents collaborate and integrate their efforts to address other complex tasks effectively. For example, the agent tasked with querying the model not only collaborates with agents responsible for semantically comparing requirements for parent/child relationships, but it also works with an agent that compares requirements to part objects to find and correct missing relationships.

Table **1** lists the different agents discussed in this paper.

Table 1. Agents and Respective Functions

Agent	Function
Query Agent	Calls upon a library of model query functions to get common model metrics quickly and easily. Queries include: • Get all requirements • Get all requirements related to **x** (ex. Power) • Get all orphaned elements • Get all model objects • How many requirements lack **x** attribute? • (i.e., validation method, test procedure, etc.) • How many requirements have **x** as its verification attribute? • (i.e., Test, analysis, demonstration, inspection etc.) • How many requirements that have 'test as its verification attribute have a test procedure defined?
Requirement Agent	Semantically compare requirements for parent/child relationships and allow user to choose if they want the agent to automatically generate the parent/child relationship.
Relationship Agent	Compare requirements to part objects for missing relationships and allow user to choose if they want the agent to automatically generate the relationship.

Each agent was provided a specific set of criteria to evaluate whether to make a recommendation. For example, Table 2 details the verbatim evaluation criteria for the relationship agent when evaluating if a relationship between a requirement and an object should be present, and Table 3 details the verbatim criteria for the relationship agent as used in the evaluations of these methods. Under the *Important Rules* category, the user explicitly instructs the SLM to recommend a relationship if it determines that the requirement under evaluation should be satisfied by a model object based on either the current model state or a plausible model extension. This is important, because it explicitly tells the SLM to make an evaluation based on the contents of both the requirement and the part object, including reasonable additions that may not currently exist.

As an example, consider a requirement that specifies that the data processing unit (DPU) must have a minimum processing power to achieve mission objectives. If the DPU is represented in the model but a processor part object has not yet been defined, the agent may incorrectly conclude that the DPU cannot satisfy the requirement because the agent treats the absence of a processor as a deliberate design choice. Instead, it may reflect an oversight by the human modeler.

Additionally, this agent is designed to return all pairings that it deems plausible to reflect the real possibility that requirements are satisfied by multiple part objects.

Table 2. Evaluation Criteria for Relationship Agent

Instructions	Description
Prompt	You are a SysMLv2 requirements hypothetical reasoning agent.
Task	Decide whether a system object (a part definition) could reasonably satisfy a given requirement within the scope of the current system architecture.
Important	• If the object already has the necessary role or immediate attributes to satisfy the requirement, answer *yes.* • If the object could plausibly gain the necessary role or attributes through small, natural extensions (e.g., adding a port, assigning an attribute that fits the subsystem's role), answer *yes*. • If the requirement is unrelated to the object's purpose, structure, or capabilities, answer *no.* • If the requirement could be satisfied only by unreasonable changes or with no clear connection, answer *no.*
Decision Rules	• If the object could logically integrate or provide the capability the requirement describes -> *yes*. • If the requirement could be satisfied only by unreasonable changes or with no clear connection -> *no.* • If the object could satisfy the requirement with a sub-part extension, but the domain match is weak -> *no*. • Prefer *no* in most cases unless there is a clear, justifiable link.

Table 3. Evaluation Criteria for Requirement Agent

Instructions	Description
Prompt	You are an expert SysMLv2 requirements reasoning agent.
Task	From the provided (pre-filtered) candidate parents, pick the single most appropriate parent for the given orphan requirements following the exact procedure below.
Procedure	1. If the orphan is top-level (target system == model), consider only candidate parents whose target system == model. 2. Structural containment (strongest evidence): • Prefer any parent where: • Parent's target system == orphan's target system, OR • Parent has a *satisfies* object whose part definition contains a subpart/type whose tokens overlap with the orphan's target system or text. • If one or more parents satisfy (a) or (b), restrict to that set. 3. Textual domain match (secondary evidence): • From the current set, prefer parents whose requirement text shares domain tokens with the orphan text. Domain tokens may include numerical units (W, kg, s) and other engineering domain words inferred from the texts. • If multiple parents remain, rank by: • Amount of token overlap with orphan text (higher is better). • Parent text is more general (shorter or higher-level wording) while orphan text is more specific. 4. Penalize but do not eliminate same-target overlap: • If candidate.same_target_overlap == true (parent and orphan target system share words), deprioritize that parent unless no other structurally/textually valid options exist. 5. Tie-breaker: • Prefer parents that are satisfied by objects closer to the orphan in part containment (explicit part/subpart ancestor). • If still tied, return the parent with the highest token overlap score. 6. Final fallback: • If no parent has any structural/textual evidence, return parent=null with confidence 0.00.
Important Rules	• If the object could logically integrate or provide the capability the requirement describes -> *yes*. • If the requirement could be satisfied only by unreasonable changes or with no clear connection -> *no.* • If the object could satisfy the requirement with a sub-part extension, but the domain match is weak -> *no*. • Prefer *no* in most cases unless there is a clear, justifiable link.

Test Methodology

A simplified SysML v2 CubeSat model was created to test and validate the agents' functionality. The agent evaluation consisted of two main parts. The first part addresses the agents' abilities to correctly and consistently evaluate and recommend the parent to child requirement relationships as well as the requirement to object relationships compared to a systems engineer's design. The second part of the agent evaluation compared the agents' output to four systems engineers of different experience levels to compare the agent's capability against humans' correctness and speed.

Table 4 is a representation of twelve example requirements written by a human. Three tests were given to the agentic tool to understand its ability to scale to more complex system models. A subset of 4 requirements, 8 requirements, and then all 12 requirements were fed into the agent to measure its ability to recommend the correct parent requirements and relationships to model components. Table 5 outlines the parent to child requirement relationships and respective part objects for each requirement per the human modeler's design.

REQ_009 through REQ_012 were intentionally written as *the system shall* instead of *the data processing unit shall*, to see if the agent could find a good parent from the model content without being provided the parent subsystem in the requirement itself.

Unlike the other objects, *DPUstates* is a behavior, and was used to verify that the model could successfully discern the different between when a *satisfied by* relationship and a *verified by* relationship should be recommended to the user.

Table 4. Example Requirements in Model

Reg ID	Requirement Name	Text
REQ_001	system_pwr	The CubeSat shall generate, store, and distribute sufficient electrical power to support all mission operations for a minimum of 6 months.
REQ_002	system_dataops	The CubeSat shall perform all data processing on-board.
REQ_003	avgpwer	The EPS shall generate at least 20W average power in Low Earth Orbit (LEO).
REQ_004	eclipseop	The electrical power system (EPS) shall store sufficient energy to operate the CubeSat in eclipse for at least 30 minutes.
REQ_005	pwrdist	The EPS shall distribute regulated power to all subsystems within specified voltage tolerances.
REQ_006	solar_eol	The solar array shall have a minimum end-of-life (EOL) power output of 22W in LEO.
REQ_007	solareff_bol	The solar cells shall achieve at least 28% efficiency at beginning of life (BOL).
REQ_008	DataProcessingUnitRequirement	The system shall have a data processing unit.
REQ_009	RAMRequirement	The system shall have a computer chip with at least 8 GB of RAM
REQ_010	NVIDIAChipRequirement	The data processing unit shall use an NVIDIA chip.
REQ_011	SSDRequirement	The system shall use a solid-state drive (SSD) for storage.

REQ_012	DPUStatesreq	The system shall power on and operate for 5s, then power down.

The *Flight System* is defined as a system consisting of five subsystems, or part definitions:

- Data Processing Unit
  - Part Objects:
    - GPU: Graphical processing unit
    - SSD: Solid-state drive
    - PSU: Power supply unit
    - RAM: Random Access Memory
    - Motherboard
  - Behavior
    - DPUstates
- Power Subsystem
  - Part Objects
    - Solar Array
    - Batteries
    - AD converter (DC-DC voltage converter)
- Thermal Management Subsystem
- Propulsion Subsystem
- Navigation & Altitude Control

Table 5. Requirement Flow and Respective Part Objects

ID	Parent ID	Level	Req Name	Satisfied By
REQ_001		0	systempwr	Power Subsystem
REQ_002		0	system_dataops	Data Processing Unit
REQ_003	REQ_001	1	avgpwr	Power Subsystem
REQ_004	REQ_001	1	eclipseop	Power Subsystem: Batteries
REQ_005	REQ_001	1	pwrdist	Power Subsystem: ADC converter
REQ_006	REQ_003	2	solar_eol	Power Subsystem: Solar Array
REQ_007	REQ_003	2	solareff_bol	Power Subsystem: Solar Array
REQ_008	REQ_002	1	DataProcessing UnitRequire-ment	Data Processing Unit
REQ_009	REQ_008	3	RAMRequire-ment	Data Processing Unit: RAM
REQ_010	REQ_008	3	NVIDIAChipRe-quirement	Data Processing Unit: GPU
REQ_011	REQ_008	3	SSDRequire-ment	Data Processing Unit: SSD
REQ_12	REQ_008	3	DPUStatesreq	Data Processing Unit: PSU
				DPUstates (This is a behavior, must be a *verified by* relation-ship)

The first portion of agent evaluation was based on two evaluation criteria, accuracy and consistency as defined below.

Accuracy: The model's recommendations align with the systems engineer's objectives.

Consistency: The agents consistently produce the same output when given the same input, with only slight semantic variations in the prompts.

Requirement agent consistency: The requirement agent was provided the following prompts to evaluate the consistency of its output.

- Can you derive the parent requirement hierarchy?
- Generate a top-level requirement tree from these subsystem needs.
- How does the child requirement trace upwards to the system-level requirements?
- Create a breakdown of functional requirements into their derived sub-requirements.
- Build a hierarchical decomposition of all the model's requirements.
- Show the derivation chain from the system-level down to technical requirements.
- Map parent and child requirement relationships for the system.
- Which high-level requirements lead to these detailed design constraints?
- Organize these requirements into a structured hierarchy showing derivations.
- Can you infer and list any derived requirements based on these user needs?

Relationship agent consistency: The relationship agent was provided the following prompts to evaluate the consistency of its output.

- Which system components currently satisfy or verify all the requirements?
- Find and link the model objects that fulfill or verify each design requirement.
- Identify the architecture elements that satisfy and verify these system-level requirements.
- Show which parts or test cases provide satisfaction and verification for the following requirements.
- Determine if all functional requirements are satisfied or verified by corresponding parts.
- Create both *Satisfied By* and *Verified By* relationships between these requirements and their components.
- Map each requirement to the subsystem that implements and verifies it.
- Analyze which components fulfill the listed requirements.
- Link the physical parts that satisfy or confirm compliance with each requirement.
- Satisfy and verify the requirements by existing model elements.

Results and Observations

Requirement Agent Evaluation: See Table 7 for results.

Consistency: The agents produced consistent output every time it was asked a different version of the same query referenced in test methodology.

Accuracy: The requirement agent achieved an average correctness of 86%. It also verified that the agent was able to successfully identify the only parent-child relationship defined in the model between REQ_008 and REQ_009. This relationship was made to test if the agent would be able to correctly identify that REQ_009 already has a parent. It is included in the Results Table 7 to verify the agent could still correctly identify its parent if the relationship hadn't been created. The agent was also able to identify that even though REQ_008 had a child, it did not have a parent and should therefore be included in its assessment.

The agents also identified parent-child relationships that agreed with the systems engineer for 10 of 12 pairings, including top level requirements (REQ_001 and REQ_002) that did not have parents.

The agent provided an incorrect assessment for REQ_004 and REQ_005. Both requirements were related to system power, and originally designated as children of REQ_001, but the agent suggested REQ_004 could be a child of REQ_008, *The system shall have a data processing unit*, and REQ_005 a could be a child of REQ_003, *The EPS shall generate at least 20W average power in Low Earth Orbit (LEO).*

The confidence score for the incorrect assessment of REQ_004 is low, only 0.50 compared to the rest of the good assessments. It appears that while the AI model has correctly identified that an SSD is a type of *storage* device, it incorrectly reasoned that its storage capability is related to the same task referenced in REQ_004 (shall *store* sufficient energy) and therefore incorrectly chose REQ_008 as its parent.

When assessing the rationale for the incorrect assessment of REQ_005, it is reasonable to understand why it chose REQ_003 as its parent instead of REQ_001. While REQ_003 and REQ_005 are intended to be siblings, without additional context, it is reasonable to imagine a reality where REQ_003 is the parent of REQ_005. Even though the assessment was incorrect, the systems engineer has deemed this a reasonable recommendation. Although incorrect, the agent flagged this requirement as an orphan in need of a parent. It also signaled to the systems engineer that the requirement may need clarification to better convey its intention.

In summary, the agent provided 10 correct parent/child recommendations, 1 reasonable recommendation, and 1 incorrect recommendation.

Relationship agent Evaluation: See Table 8 for results.

Consistency: The agents produced consistent output every time it was asked a different version of the same query referenced in test methodology.

Accuracy: The relationship agent successfully provided correct relationship recommendations for all 12 requirements. For REQ_001, REQ_002, and REQ_003, the agent generated two recommendations each. In these cases, the first recommendation for each requirement was accurate, while the second recommendation was not deemed a good match. The agent was also able to identify that DPUstates was a behavior and successfully recommended a *verify by* relationship between REQ_012 and the DPUstates behavior object.

This behavior is considered reasonable for the agent, as its primary objective is to suggest relationships for any match it deems plausible. By design, the agent prioritizes generating a broader set of recommendations, even if some may not be ideal, rather than omitting potential matches that could be suitable. This approach aligns with the preferred behavior of ensuring comprehensive coverage of possible relationships.

Additionally, this behavior reflects the scenario in which a requirement may have multiple valid relationships with different objects. Relationships of this nature are not always one-to-one, and the agent's ability to account for this complexity enhances its utility in identifying potential matches.

Observations: The results of this experiment were promising, exhibiting an average correctness of 86% for parent-child requirement recommendation, and almost 96% correctness for requirement-object recommendation. While there were also instances of agents recommending relationships that the human SME did not agree with, it still provided useful insights to the user that the human SME did not even notice at first glance.

For example, in one of the initial agent tests, the agent recognized that the DPU did not have an SSD part object defined but reasoned that it is plausible one could exist and therefore recommended a reasonable *satisfy by* relationship between REQ_011 and the DPU. This was an unexpected positive outcome. The human SME did not even realize that the DPU was missing an SSD part object. Therefore, the agent not only successfully recommended the reasonable relationship between

requirement and part object but also recognized that the DPU part object should have an SSD part object created.

Human Comparison: The agent's results were then compared to systems engineers. The findings presented in this paper are exploratory in nature and do not claim statistical significance due to the limited sample size.

Two senior-level engineers (>10 years of professional experience) and two junior-level engineers (<5 years of professional experience) were chosen to participate in a similar test the agent was given. Three sets of L0-L2 requirements in an excel file were provided containing 4, 8, and 12 requirements, respectively. The participants were asked to provide the parent requirement for each requirement in each set and then provide the relevant subsystem that satisfies each requirement. No additional context was provided. See Figure 3 for the average timed results for the requirement test and see Figure 4 for the average timed results for the relationship test.

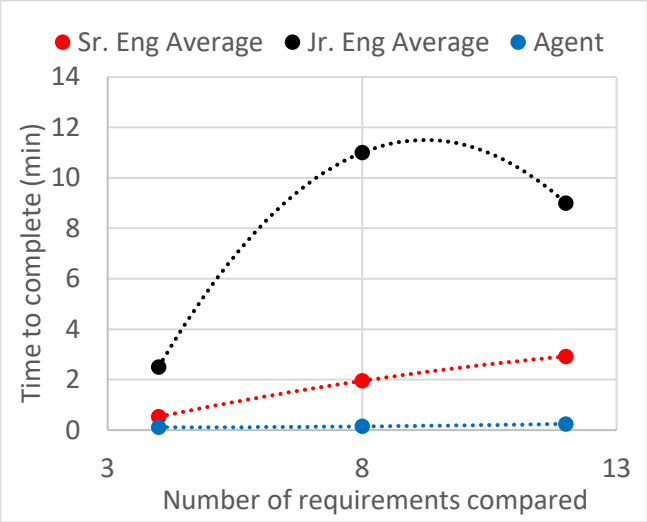


Figure 3. Requirement Agent Average Time Results

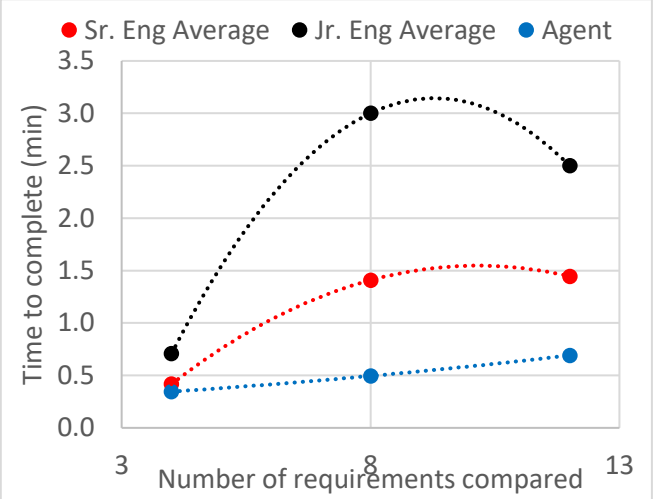


Figure 4. Relationship Agent Average Time Results

The requirement agent was approximately 10 times faster (0.25min) than the fastest systems engineer (2.75 min) for the 12-requirement test. While the relationship agent was only four seconds faster (0.69 min) than the fastest systems engineer (0.75 min), unlike the agent, the human did not have to search a model to find and make the recommended relationships, and the fastest engineer was the most senior. These timing results are consistent with previous bodies of work that demonstrate that agents are faster at tasks requiring text output.

See Table 6 for the agent correctness results. By averaging the requirement agent results, the humans have an overall average score of 77% correctness, and the agent has an 86% average. If the junior engineers are excluded from the average, the results between human and agent become comparable in value. The results for both the relationship agent and human were also comparable. The results presented represent simple averages of the test scores and are not intended to imply statistical significance. Additionally, this test used a simplified model of only 12 requirements, and it is recognized that real systems can have thousands.

Table 6. Human vs. Agent Results (Fraction Correct: Requirement Agent, Relationship Agent)

Test	Jr. Eng1	Jr. Eng2	Sr. Eng1	Sr. Eng2	Agent
Test 1: 4 reqs.	2/4, 4/4	3/4, 3/4	4/4, 4/4	4/4,4/4	4/4, 4/4
Test 2: 8 reqs.	7/8, 8/8	5/8, 8/8	5/8, 8/8	7/8, 8/8	6/8, 7/8
Test 3: 12 reqs.	8/12, 12/12	7/12, 12/12	9/12, 12/12	10/12, 12/12	10/12, 12/12

Not only are the agents faster than the tested engineers, but also, they are comparable in accuracy and

correctness. Even though the agentic tool demonstrated faster performance and comparable correctness to systems engineers in the tested scenarios more evaluation is needed, as the systems engineers were not provided with additional context or access to a model, which could have influenced their performance. Meanwhile, the agent had a SysMLv2 simplified model with parts and relationships to leverages for search and context. While having access to a model aid in the agent's correctness, it also increases its speed since it searches and retrieves information to make more efficient comparisons. Similarly, if the humans had a model to leverage, it may have increased the correctness of their responses, but it would have decreased their speed as search and retrieval of model information is often time-consuming.

Additionally, the plateau (or even decrease) in completion time from test 2 to test 3 for some of the systems engineers may be attributed to the humans becoming more familiar (and therefore faster) with the test format. However, it is assumed that as the number of tasks increases, the human time will eventually scale with the complexity.

Like the agent's incorrect responses, the systems engineers' incorrect responses were reasonable, in that without additional project context, their answers were deemed plausible. While not ideal, this also may point to poorly written requirements or gaps in the model that the human should address. Even though REQ_009 through REQ_012 were intentionally written ambiguously, the agent was able to discern the differences, but some systems engineers did not.

Conclusion and Future Work

Conclusion

A multi agentic tool for SysML v2 model validation was developed with promising initial results. While the results presented in this paper were obtained using a supercomputing center, this approach was primarily chosen to protect sensitive data by ensuring all operations remained within an organization-approved network. However, this tool is also capable of running on standard hardware (16GB of RAM and 1TB of storage) with open-source Ollama tools because the small language model used in this application is lightweight enough to be deployed locally.

The agentic framework combines deterministic and probabilistic methods, giving users control over SLM decisions through objective rules embedded in agents. It also captures systems engineering expertise to address common semantic and syntactic challenges.

The agentic tool also utilizes a graph database to overcome the conventional context window limitations of commercial LLM chatbots for more efficient search and retrieval. This way, the SLM only receives the relevant information needed to make a decision. This lowers the total compute power required while also reducing potential for error because superfluous information is omitted from the query.

While there were some areas where the agent provided incorrect recommendations, the agent was able to provide insights, explain itself, and point out where parts of the model may need additional review or clarification, which was the goal of this agentic tool. This agentic tool was also tested against human performance by providing both the agent and human modelers with similar tests. The agent was consistently faster than the humans who performed the test, while also achieving comparable accuracy.

Future Work

This research introduces an initial framework for leveraging AI in SysML v2 model validation, demonstrating promising results on a simplified CubeSat model. Further research and development are necessary to integrate the framework with industry-standard tools, such as MagicDraw, and to evaluate its capabilities on large, real-world models.

The modular architecture of this framework offers significant potential for expansion. Existing agents may be refined to become more sophisticated, new agents added to address new validation objectives, and provide new design recommendations. This modularity supports the development of a comprehensive end-to-end agentic modeling application. Examples of future capabilities include assessing requirement quality, identifying incompatible interfaces, and handling complex metric queries. Additionally, enhanced search and retrieval rules may be incorporated to improve efficiency and speed.

This work represents a step forward in applying AI to systems engineering, enabling future research to fully leverage AI verification of MBSE models.

Table 7 Requirement Agent Output ('✓'-Rationale Aligns with Human Assessment, X-rationale Deemed Incorrect by Human)

Orphan ID	Parent ID	Confidence	Rationale	
REQ_001	(no parent found)	0.00	There is no feasible parent requirement for this orphan because its target system points towards the top-level of the model (CubeSat).	✓
REQ_002	(no parent found)	0.00	There is no feasible parent requirement for this orphan because its target system points towards the top-level of the model (CubeSat).	✓
REQ_003	REQ_001	0.60	The parent *system_pwr* structurally contains the orphan requirement as it has a *satisfies* object whose part_definition contains a subpart named *electrical power system (EPS)*, which matches the target_system of the orphan requirement. Additionally, there is textual overlap between the two requirements, with both discussing electrical power for the CubeSat.	✓
REQ_004	REQ_008	0.50	The *DataProcessingUnitRequirement* parent is selected because it structurally contains the orphan requirement within its satisfied_by_part object. The subpart *SSD*, a type of storage system, partially overlaps with the target system of the orphan *(electrical power system (EPS))*. Additionally, the textual domain overlap between the two requirements is moderately strong, as both involve the concept of storage and electrical power.	✗
REQ_005	REQ_003	0.70	The parent *avgpwer* satisfies structural containment evidence as it has a *satisfies* object whose part_definition contains a subpart *electrical power system (EPS)*, which exactly matches the orphan's target_system. Additionally, *avgpwer* shares textual domain tokens with the orphan such as *power*, *system* and *voltage tolerances*. The parent *avgpwer* is more general in its wording compared to the orphan, making it a good match.	✗
REQ_006	REQ_003	0.60	The parent *avgpwer* structurally contains evidence through part satisfaction: it shares a common target system *(electrical power system (EPS))* and its requirement text mentions an average power output of at least 20W in Low Earth Orbit (LEO), which overlaps with the orphan's requirement for a minimum end-of-life (EOL) power output of 22W in LEO.	✓
REQ_007	REQ_003	0.60	The parent *avgpwer* structurally contains evidence of a match with the orphan requirement through its satisfied_by_part object, which includes a subpart named *solar array*. Additionally, there is a strong textual/domain overlap between the two requirements due to the mention of *power*, *electrical*, and *efficiency*. However, it's important to note that the orphan requirement specifically mentions *solar cells*, while *avgpwer* refers to *solar array*. While there is some difference in specificity, this overlap still provides a reasonable match.	✓
REQ_008	REQ_002	0.60	The parent *system_dataops* matches structurally as it shares a subpart with the orphan requirement (CubeSat). Additionally, there is moderate textual overlap between the two requirements, with both mentioning data processing. However, no direct mention of Data Processing Unit is found in *system_dataops*, which makes the confidence score somewhat lower.	✓
REQ-009	REQ-008	0.60	The parent 'DataProcessingUnit' structurally contains the orphan requirement as a subpart within its part definition. The 'DataProcessingUnit' has attributes 'Mass' and 'electricalPower', which overlap with the 'RAM' and power-related terms in the orphan requirement.	✓
REQ_010	REQ_008	0.60	The parent DataProcessingUnitRequirement has a strong textual domain overlap with the orphan requirement as both texts discuss the 'system' and data processing unit. However, no structural containment evidence is found between the two requirements directly. The DataProcessingUnitRequirement satisfies a part named DPU, which has a subpart/type named GPU that might be inferred to include NVIDIA chips. This is the best match from the provided list of candidates.	✓
REQ_011	REQ_008	0.60	Structural containment evidence was found for parent DataProcessingUnitRequirement. In its part_definition, the storage subpart has a type of *SSD*, which overlaps with the orphan requirement's target_system (system). This satisfies condition (b) from the procedure. Textual domain overlap is also present as both requirements mention the words system and storage. The parent's requirement text is more general than the orphan, making it a stronger match.	✓
REQ_012	REQ_008	0.60	The orphan DPUStatesreq requires a system to power on and operate for 5s, then power down. This is related to the data processing unit since the time duration is relevant for computational tasks. Structurally, DataProcessingUnitRequirement satisfies this orphan as it has a part definition that includes DPU, which in turn contains subparts like GPU, RAM, and CPU. Since these subparts don't have specific power-on/off requirements, the orphan is more related to the overall DataProcessingUnitRequirement rather than any specific subpart.	✓

Table 8 Relationship Agent Output ('✓'-Rationale Aligns with Human Assessment, X-rationale Deemed Incorrect by Human)

Req ID	Part	Relationship	Confidence	Rationale	
REQ_001	PowerGenera-tionSubsystem	Satisfy	1.00	The PowerGenerationSubsystem is explicitly designed to generate, store and distribute electrical power (as indicated by its attributes and subparts). It contains solar panels, batteries, and an AD converter which are all components that contribute to power generation, storage, and distribution. The system architecture has clear structural evidence supporting the satisfaction of REQ-001.	✓
REQ_001	FlightSysDef	Satisfy	0.70	The FlightSysDef is responsible for various system functions, including power management in a CubeSat context. Therefore, it logically integrates the capability to generate, store, and distribute electrical power to support all mission operations.	✗
REQ_002	DataProcessing Unit	Satisfy	1.00	The DataProcessingUnit is designed to perform data processing tasks as indicated by its purpose and the presence of ports (ac_in, cmd) for external data input/command. Its subparts include CPU, GPU, RAM, and storage which are typically used in data processing.	✓
REQ_002	FlightSysDef	Satisfy	0.40	The FlightSysDef is a part of the CubeSat's architecture, and while it doesn't explicitly have data processing capabilities, it could plausibly gain them through small extensions such as adding necessary ports for data input/output or integrating processing sub-parts. However, the confidence is moderate due to the absence of specific details about the FlightSysDef's current capabilities and the potential need for additional hardware or software.	✗
REQ_003	PowerGenera-tionSubsystem	Satisfy	0.70	The PowerGenerationSubsystem has the necessary role of power generation and contains components such as solar panels and batteries which can generate electrical power. The AD converter is also present which can convert the generated power into a usable format. Additionally, there is no need for unreasonable changes or extensions to satisfy this requirement.	✓
REQ_003	PropulsionSub-system	Satisfy	0.80	The PropulsionSubsystem is primarily responsible for generating thrust and managing the vehicle's motion. However, it also indirectly contributes to electrical power generation through the propulsion system's auxiliary power unit (APU), which generates electricity to support various subsystems. In Low Earth Orbit (LEO), the main function of the APU is typically to provide power for onboard systems. Given that the APU can generate more than just 20W, it can reasonably be assumed that the PropulsionSubsystem could generate at least 20W average power in LEO.	✗
REQ_004	PowerGenera-tionSubsys-tem.batteries	Satisfy	0.70	The PowerGenerationSubsystem contains batteries, which can store energy. If the capacity of these batteries is sufficient to provide power for the CubeSat during eclipse as per the requirement, then this subsystem could satisfy the 'eclipseop' requirement. The connection between the requirement and the PowerGenerationSubsystem is strong structurally.	✓
REQ_005	PowerGenera-tionSubsys-tem.ADcon-verter	Satisfy	0.70	The PowerGenerationSubsystem is responsible for power generation in the system architecture and includes parts such as solar panels and batteries which produce power. Furthermore, it also contains an AD converter which might be used for regulating or distributing electrical signals, although not explicitly stated as 'power' per se. Given these components and their roles within the subsystem, it is plausible that the PowerGenerationSubsystem could satisfy the requirement REQ-005 to distribute regulated power to all subsystems within specified voltage tolerances.	✓
REQ_005	DataProcessing Unit.PSU	Satisfy	0.60	The DataProcessingUnit (DPU) includes a Power Supply Unit (PSU), which is responsible for providing electrical power to the sub-parts within DPU. Although the specific regulation of power distribution across all subsystems (not just DPU's sub-parts) might require additional control mechanisms, it's plausible that such mechanisms could be integrated into the PSU or added as a new sub-part in a small and natural extension to the system.	✗
REQ_006	PowerGenera-tionSubsys-tem.solar-panels	Satisfy	0.70	The 'PowerGenerationSubsystem' includes the *solarpanels* subpart, which is a type of solar array. Given that there is a logical integration and clear connection between the solar panels and the requirement of having a minimum EOL power output, it is reasonable to conclude that the PowerGenerationSubsystem could satisfy the REQ-006 could satisfy the REQ-006.	✓
REQ_007	PowerGenera-tionSubsys-tem.solar-panels	Satisfy	0.70	The 'PowerGenerationSubsystem' contains the *solarpanels* sub-part which is responsible for generating power from solar energy. The requirement states that the solar cells shall achieve at least 28% efficiency at beginning of life (BOL). Since solar panels are the primary components used in solar power generation systems to convert light into electricity, it is logically possible and with high confidence that they can meet this requirement.	✓
REQ_010	DataProcessing Unit.GPU	Satisfy	0.70	The DataProcessingUnit object includes a subpart named GPU, which typically refers to the processing unit inside a computer that is responsible for graphics and can be manufactured by NVIDIA.	✓
REQ_011	DataProcessing Unit.SSD	Satisfy	1.00	The DataProcessingUnit contains a subpart named 'storage' with the type 'SSD'. This matches exactly with the requirement for a solid-state drive (SSD).	✓
REQ_012	DataProcessing Unit.PSU	Satisfy	0.70	The DataProcessingUnit contains a Power Supply Unit (PSU), which is responsible for powering on and off the system. Additionally, the requirement asks for a duration of operation, which can be implemented through software control. Therefore, it's logically feasible for the DataProcessingUnit to satisfy this requirement with strong structural evidence.	✓
REQ_012	DPUStates	Verify	1.00	The behavior *DPUStates* directly simulates the system function described in requirement *DPUStatesreq*. The behavior specifies a sequence of states (on and off) that corresponds to the requirement's description of powering on and operating for 5s, then powering down.	✓

References

Belcak, P., Heinrich, G., Diao, S., Fu, Y., Dong, X., Muralidharan, S., Lin, Y. C. & Molchanov, P. 2025, Small Language Models are the Future of Agentic AI, arXiv, Available at: https://arxiv.org/abs/2506.02153

CDW. (2023, October 17). *What is on-premises IT infrastructure?* CDW. https://www.cdw.com/content/cdw/en/articles/datacenter/what-is-on-premises-it-infrastructure.html

Huizenga, E., & Yang, B. (2025, April 9). Agent development kit: Making it easy to build multi-agent applications. Google Developers Blog. https://developers.googleblog.com/en/agent-development-kit-easy-to-build-multi-agent-applications/

IBM. (n.d.). What is a context window? IBM Think. Retrieved October 20, 2025, from https://www.ibm.com/think/topics/context-window

Johns, B. & Carroll, K. & Medina, C. & Lewark, R. & Walliser, J. 2024, AI Systems Modeling Enhancer (AI-SME): Initial Investigations into a ChatGPT-enabled MBSE Modeling Assistant, INCOSE, Dublin, Ireland.

LlamaIndex. (n.d.). LlamaIndex: A Data Framework for LLM Applications. Retrieved August 26, 2025, from https://www.llamaindex.ai/

Ludovico, I., Barriga, A., Rutle, A., & Heldal, R. (2020). Model repair with quality-based reinforcement learning. The Journal of Object Technology, 19(2). https://doi.org/10.5381/jot.2020.19.2.a17

Neo4j, Inc. (2025). *Neo4j Documentation*. Retrieved October 21, 2025, from https://neo4j.com/docs/

Mistral AI. (2023, September 27). *Mistral 7B: The best 7B model to date, Apache 2.0*. Retrieved from https://mistral.ai/news/announcing-mistral-7b

Moody, D. 2005, Theoretical and practical issues in evaluating the quality of conceptual models: Current state and future directions, Elsevier, Amsterdam, Netherlands.

Morkevičius, A., Aleksandravičienė, A., & Strolia, Z. (2022). *System verification and validation approach using the MagicGrid (formerly MBSE Grid) framework*. INCOSE Insight, 26(1), 7–51.

Object Management Group. (2025). *SysML® v2: The next-generation systems modeling language!* Retrieved November 10, 2025, from https://www.omg.org/sysml/sysmlv2/

Sultan, B., & Aprille, L. (2024, September 22). Ai-driven consistency of SysML diagrams | proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems. ACM digital Library. https://dl.acm.org/doi/10.1145/3640310.3674079.

Wang, Z. Z., Shao, Y., Shaikh, O., Fried, D., Neubig, G., & Yang, D. (2025). How Do AI Agents Do Human Work? Comparing AI and Human Workflows Across Diverse Occupations.

Biography



Qiana Curcuru

Qiana Curcuru earned her BSE and MSE in Mechanical Engineering at the University of Massachusetts Lowell in 2019. Since joining MIT Lincoln Laboratory in 2019 she has served as a data and systems engineer for large space programs while also researching how artificial intelligence can make systems engineering more efficient and effective.



Ethan Cox

Ethan Cox is an undergraduate researcher at Iowa State University working at the intersection of artificial intelligence, product development, and applied systems research. He previously interned at MIT Lincoln Laboratory and will continue his work this summer at Johns Hopkins Applied Physics Laboratory. His interests focus on AI-driven products and autonomous agent systems that enhance decision-making and human—AI collaboration.